SRE lab 1 - Java Observability Pipeline

The stack: Java app (8080) > Prometheus (9090) > Grafana (3000)
Repo: `github.com/ALTMANhehe/java-sre-observability`

Core concepts:

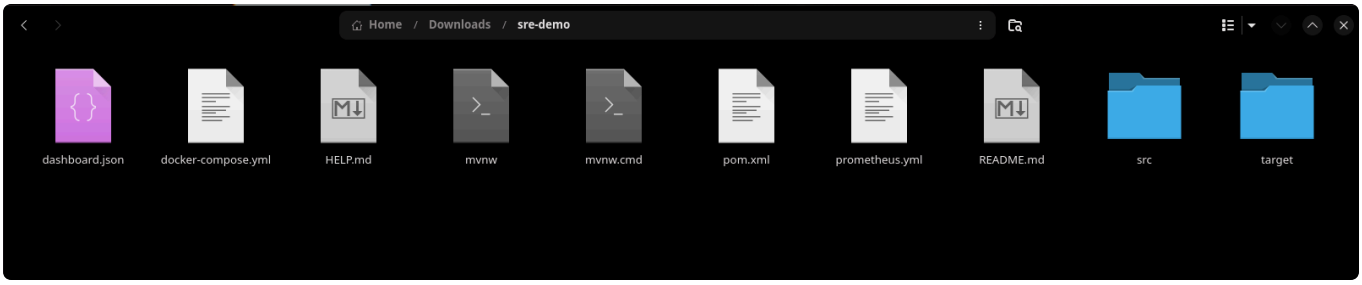
- **Micrometer/Actuator:** Turns a standard Java app into a metrics generator. (Endpoint: `/actuator/prometheus`).
- **Prometheus:** The scraper. It pulls data every 5 seconds and stores it.
- **Grafana:** The visualizer. Connects to Prometheus and draws graphs.
- **Docker Compose:** Runs the whole monitoring stack locally.
- **`network_mode: host`:** Bypasses Docker's isolated network so containers can talk to the host's `localhost`.
- **Incident Simulation:** Kill the Java app (`Ctrl+C`) → Prometheus goes RED (Target DOWN) → Grafana flatlines.

Commands to remember:

```
- Run Java app: ./mvnw spring-boot:run
```

```
- Start monitoring: docker compose up -d
```

```
- Stop monitoring: docker compose down
```



Step1: The Java App (pom.xml & application.properties)

"Add `spring-boot-starter-actuator` and `micrometer-registry-prometheus` to the pom.xml"

```
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
        <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
    </dependencies>
```



How it works:

- When the Java app starts, Spring Boot reads these files.
- The actuator library create a hiddent URL at `/actuator/prometheus`.
- The micrometer library collect all the JVM memory and CPU data and format it as plain text.

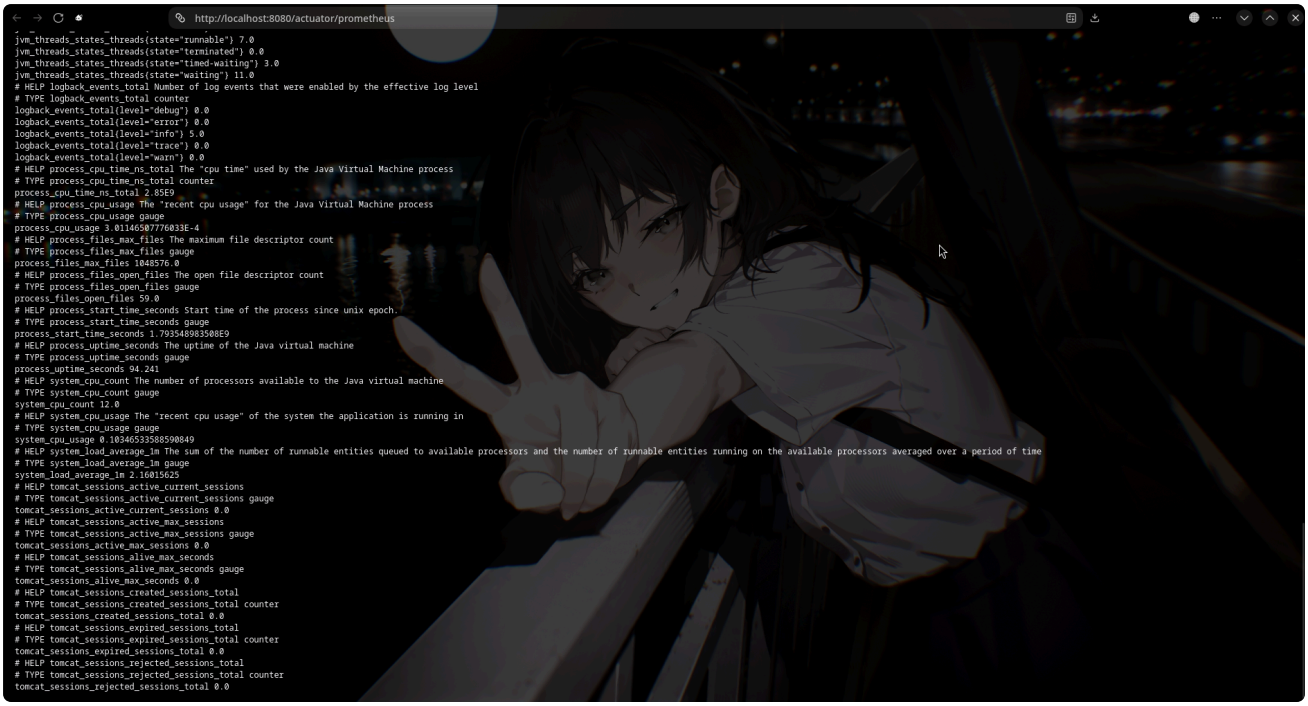
Step 2: The Scraper (prometheus.yml)

```yaml
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: 'spring-boot-app'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['localhost:8080']
```

How it works:

- When Docker starts the Prometheus container, it reads this file.
- Every 5 seconds, Prometheus acts like a browser and makes an HTTP GET request to `http://localhost:8080/actuator/prometheus`.
- This is what `http://localhost:8080/actuator/prometheus` has:



- Prometheus grabs that wall of text and saves the numbers into its time-series database.

Step 3: The Infrastructure (docker-compose.yml)

"Define `prometheus` and `grafana` as services, map their ports `9090` and `8080`, and add `network_mode: host`"

```yaml
services:
  prometheus:
    image: prom/prometheus:latest
    network_mode: host
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana:latest
    network_mode: host
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin

```

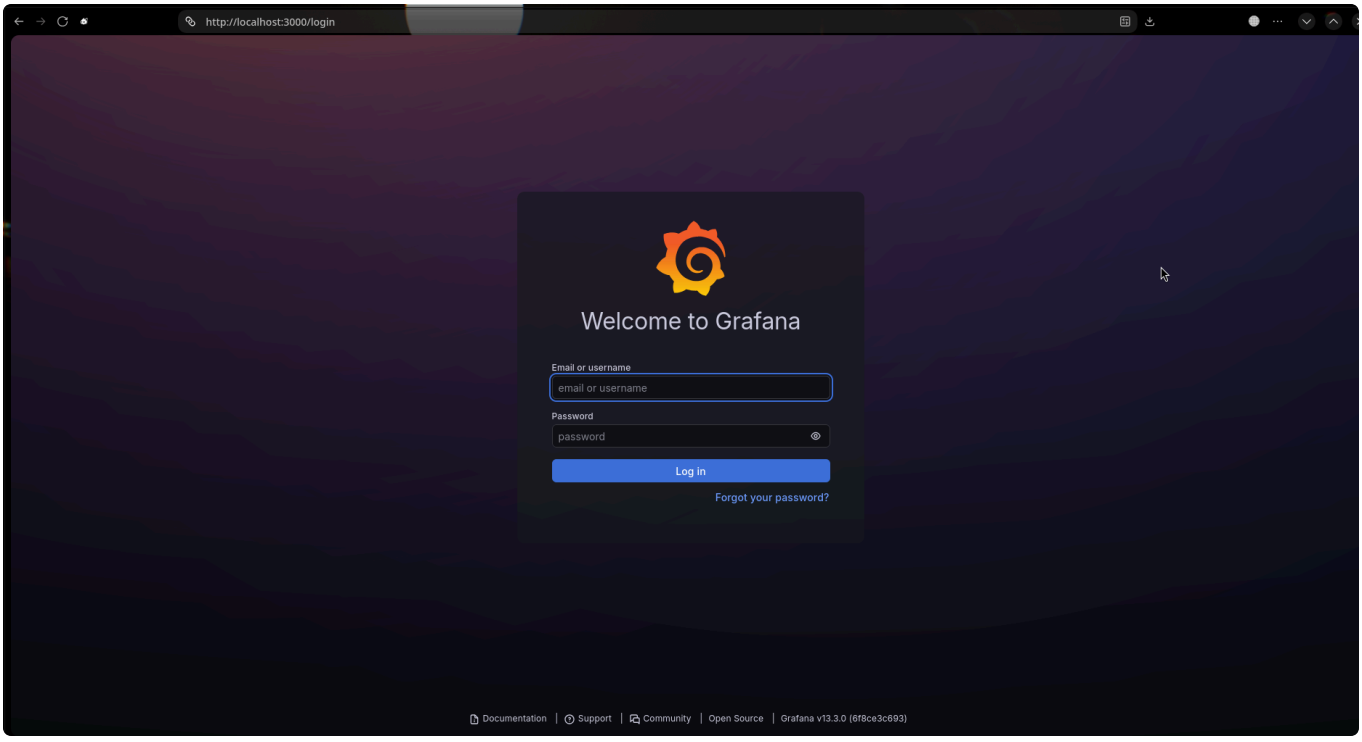
```
~/Downloads/sre-demo main
❯ docker compose up -d
[+] up 2/2
 ✔ Container sre-demo-grafana-1    Started
 ✔ Container sre-demo-prometheus-1 Started
```

How it works:

- When you ran `docker compose up -d`, Docker read this file, downloads the official images from the internet, and started them in the background.
- Because of `network_mode: host`, Docker didn't create a fake isolated network.
- It plugged both containers directly into your machine's network.
- Thats why Prometheus could reach `localhost:8080`.

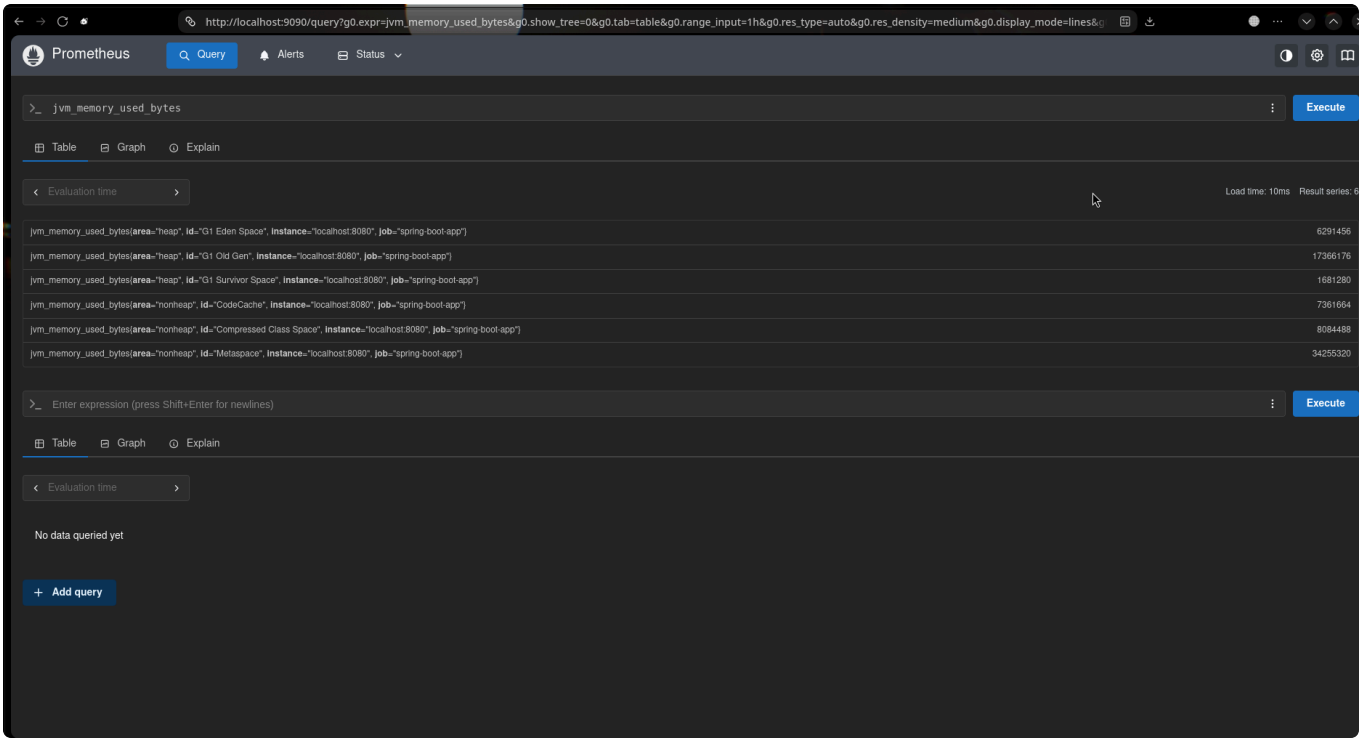
##Step 4: The Visualization (Grafana UI)

Log into Grafana `localhost:3000`

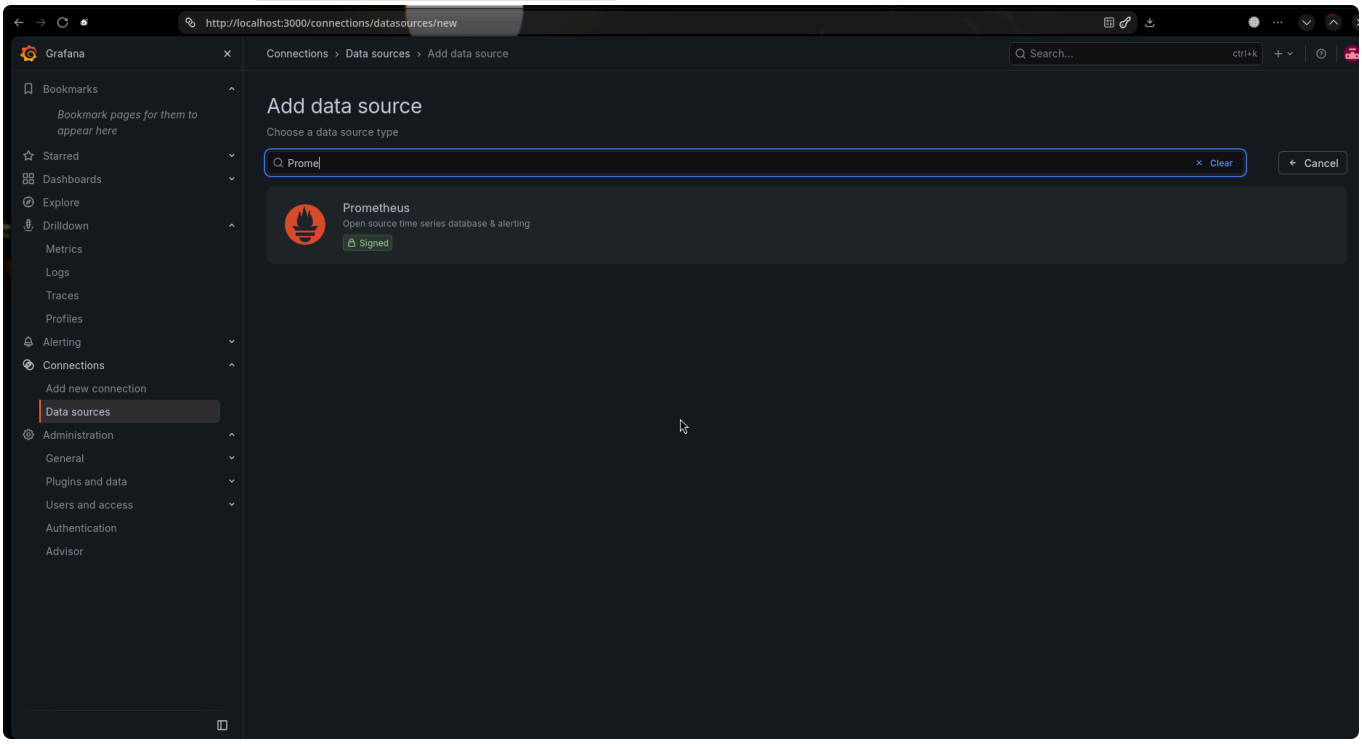


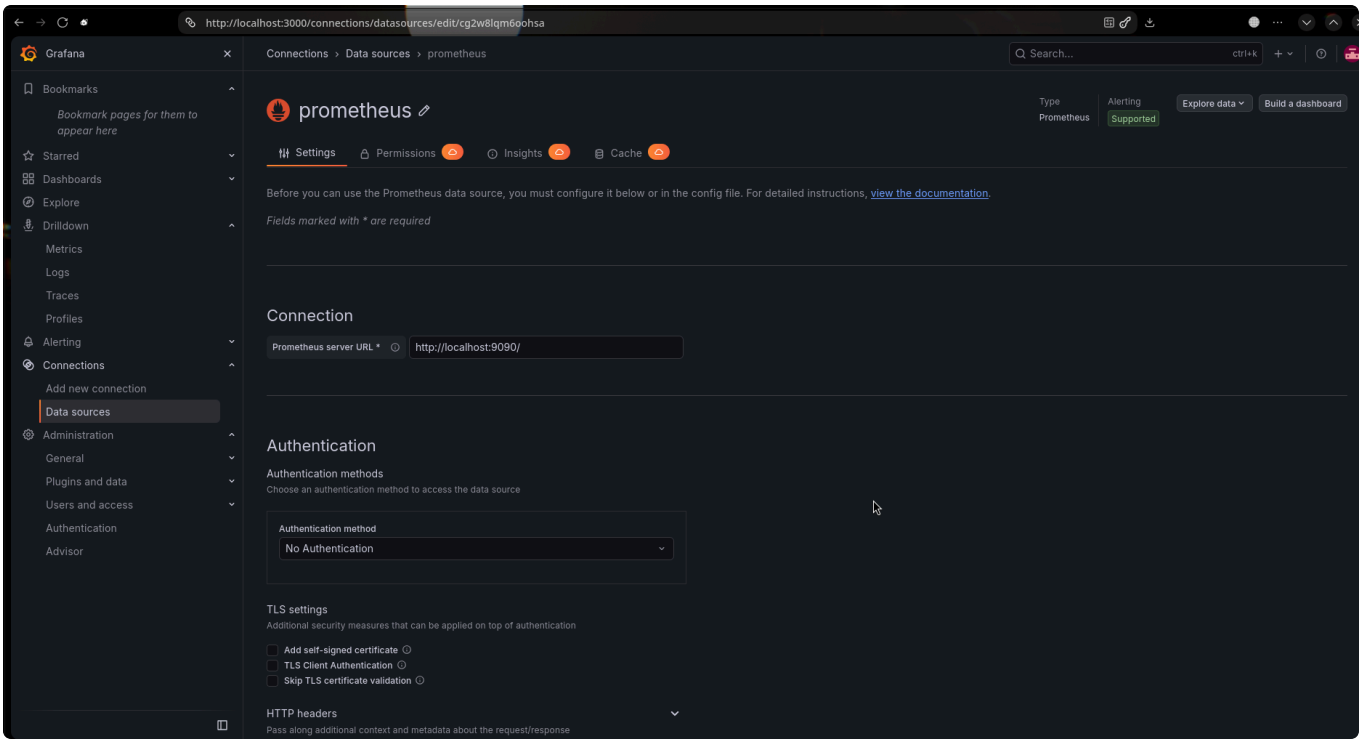
Run this query on Prometheus `http://localhost:9090`

```
jvm_memory_used_bytes
```

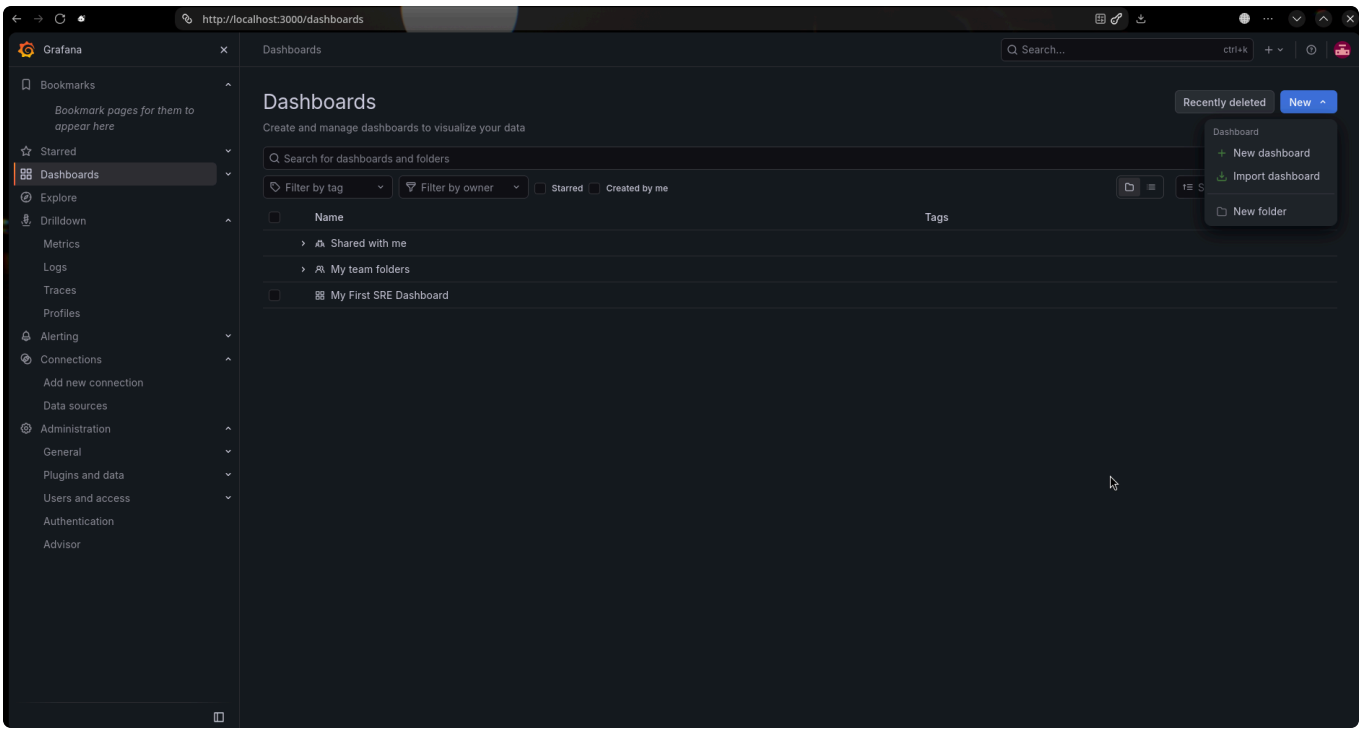


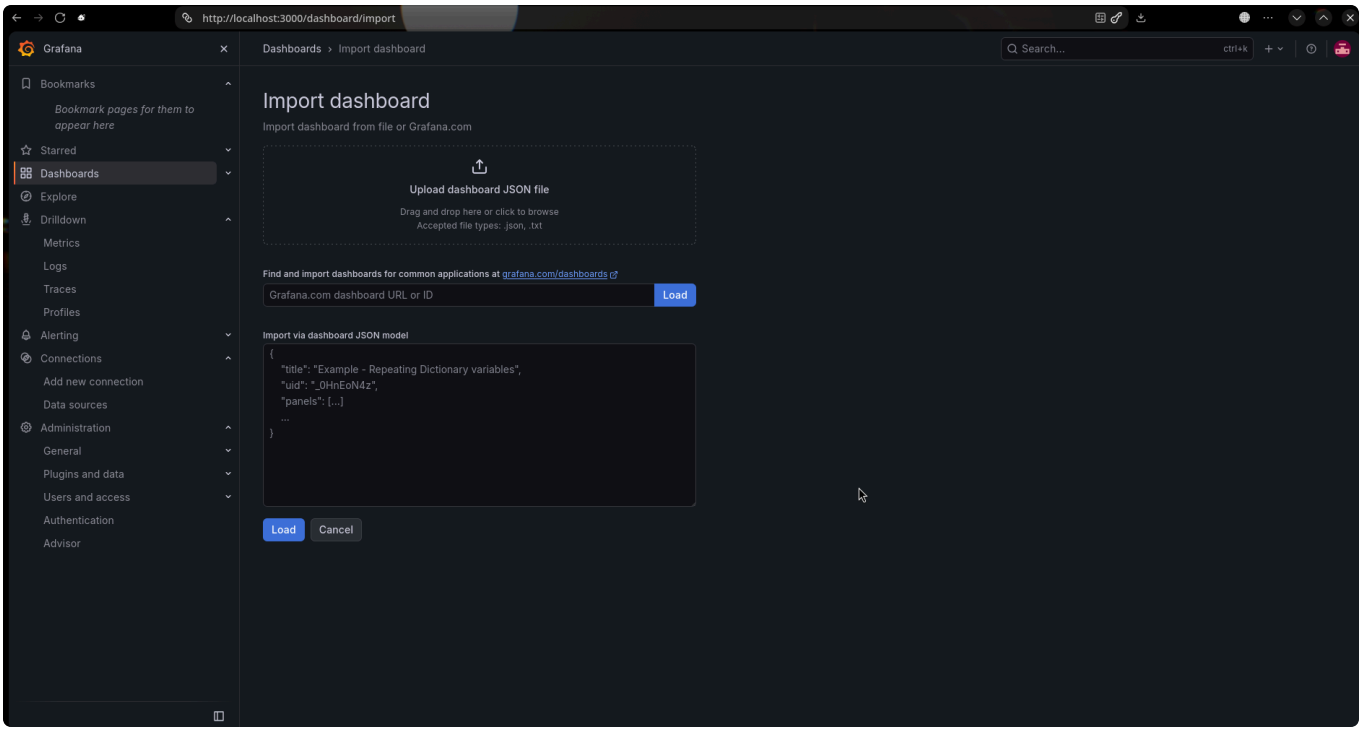
Add Prometheus `http://localhost:9090`



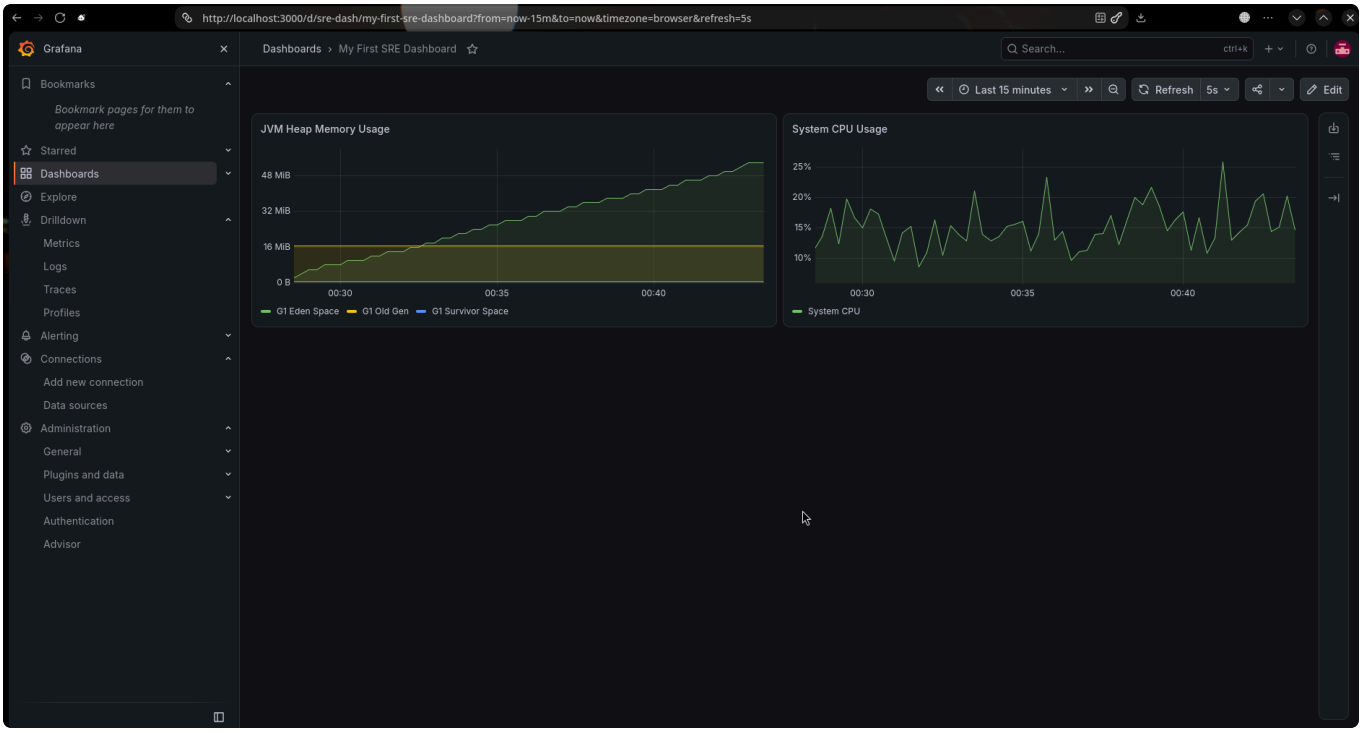


Add the Dashboard (dashboard.json)





Walah!!



How it works:

- When Grafana loaded the dashboard, it saw the JSON instruction: "Rn the query `jvm_memory_used_bytes`".
- Grafana sent an API request to Prometheus at `localhost:9090`.
- Prometheus looked at its database, returned the numbers, ad Grafana drew the lines.